

# Прогнозирование телефонной нагрузки компании Айтекс (микросервисная архитектура)

Исполнитель: Заварницын Сергей

Группа: OTUS ML Advanced-2026



sergeyzavarnitsyn



sergicz

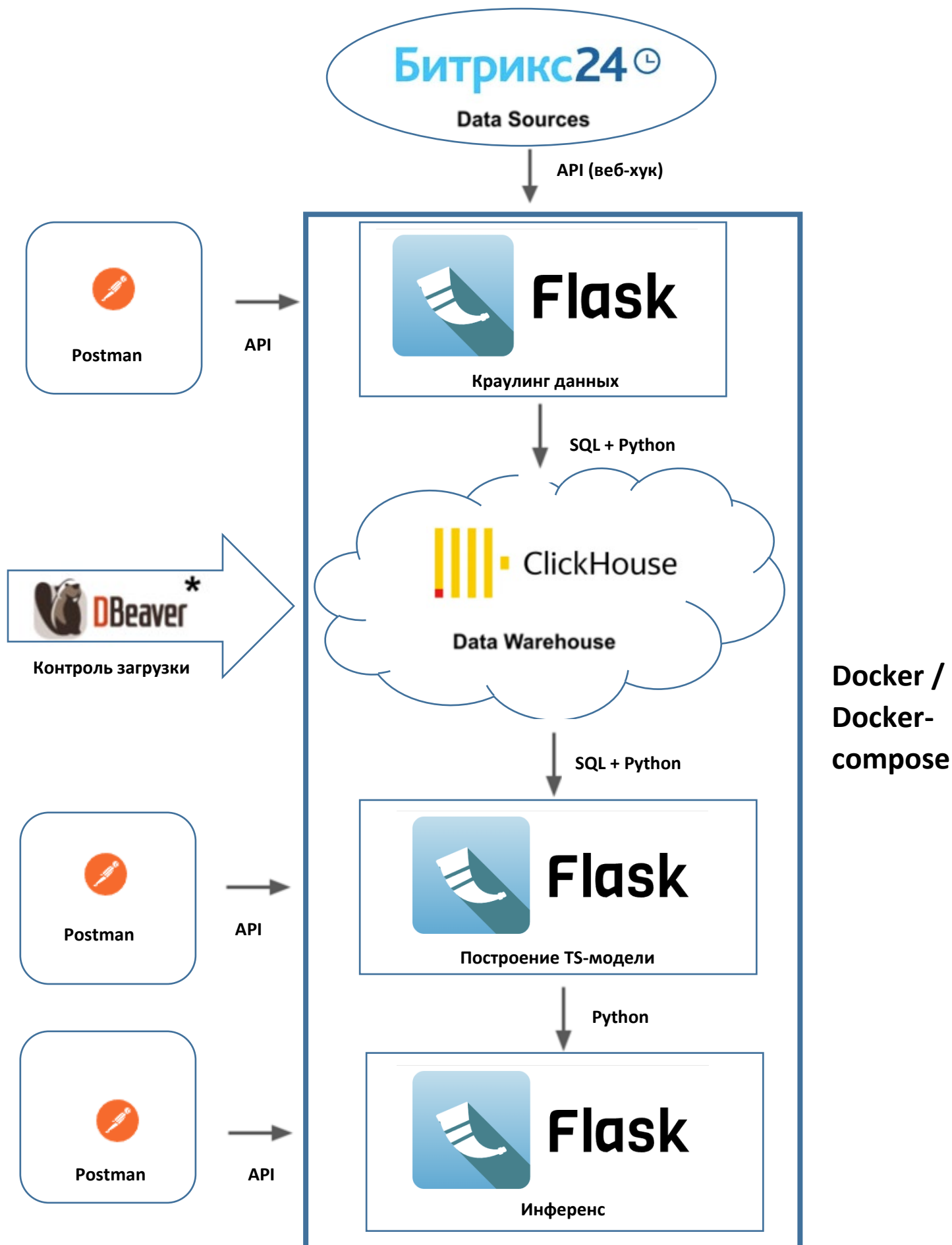
**Бизнес-цель дипломной работы** – создать и вывести в продакшн микросервисное приложение для прогнозирования телефонной нагрузки на операторов технической поддержки на любую будущую дату на основании исторических данных о совершенных звонках за предыдущие периоды. Это необходимо для планирования нужного количества операторов техподдержки, распределения отпусков, предотвращения возможного перегруза операторов и рисков недозвона клиентов.

**Предыстория** – ранее в 2023 году на курсе Machine Learning. Professional подобная задача была решена в виде Jupiter-ноутбука с применением линейных моделей и ручной генерации признаков. Текущая работа развивает заложенную ранее идею с помощью TimeSeries-моделей и микросервисной архитектуры в продовом варианте.

#### **Definition of done:**

1. Создать с помощью Docker-Compose два взаимодействующих микросервиса – хранилище данных о звонках на базе ClickHouse + Flask-приложение для непосредственного сбора данных в базу, построения модели на основании TimeSeries и предсказания количества звонков на любую дату.
2. Создать таблицы в базе данных ClickHouse с помощью DBEaver.
3. С помощью Postman или любого веб-браузера протестировать все режимы работы приложения, используя разные декораторы вызова.
4. HealthCheck Flask-приложения.
5. Проверка связи Flask-приложения с базой ClickHouse.
6. Краулинг данных о количестве звонков за предшествующий период в виде json-структур через вебхук Битрикс24 посредством функции Flask-приложения с сохранением их в базу ClickHouse.
7. Построение TimeSeries-модели посредством функции Flask-приложения с последующим сохранением модели в joblib-формате.
8. Загрузка сохраненной модели и предсказание количества звонков на любую дату посредством функции Flask-приложения.
9. Сформулировать выводы из дипломной работы и всего курса.

## Архитектура и основные компоненты системы



## Структура данных

### 1. Пример исходных данных, возвращаемых веб-хуком:

```
{
  "result": [
    {
      "ID": "1",
      "PORTAL_USER_ID": "36",
      "PORTAL_NUMBER": "LINK_BASE_NUMBER",
      "PHONE_NUMBER": "79086374391",
      "CALL_ID": "0e30b0536fc71bb1.1505801330.1254990",
      "EXTERNAL_CALL_ID": null,
      "CALL_CATEGORY": "external",
      "CALL_DURATION": "0",
      "CALL_START_DATE": "2017-09-19T11:08:53+05:00",
      "CALL_RECORD_URL": null,
      "CALL_VOTE": null,
      "COST": "0.0000",
      "COST_CURRENCY": "RUR",
      "CALL_FAILED_CODE": "603-S",
      "CALL_FAILED_REASON": "Decline self",
      "CRM_ENTITY_TYPE": null,
      "CRM_ENTITY_ID": null,
      "CRM_ACTIVITY_ID": null,
      "REST_APP_ID": null,
      "REST_APP_NAME": null,
      "TRANSCRIPT_ID": null,
      "TRANSCRIPT_PENDING": "N",
      "SESSION_ID": "119411609",
      "REDIAL_ATTEMPT": null,
      "COMMENT": null,
      "RECORD_FILE_ID": null,
      "CALL_TYPE": "1"
    }
  ]
}
```

### 2. Структура базы данных DWH Clickhouse

The screenshot shows the DBeaver 22.0.2 interface. On the left, the 'Базы данных' (Databases) tree is expanded, showing a hierarchy: 'mydatabase' > 'information\_schema' > 'itex' > 'b24' > '123 calls (Int64)'. The 'b24' table is highlighted. On the right, the 'Данные' (Data) tab for the 'b24' table is displayed, showing a table with 27 rows and 3 columns: 'id', 'date', and 'calls'. The table is filtered to show only rows where 'calls' is greater than 5. The data is as follows:

| id | date       | calls |
|----|------------|-------|
| 1  | 2023-02-07 | 6     |
| 2  | 2023-02-07 | 8     |
| 3  | 2023-02-07 | 8     |
| 4  | 2023-02-07 | 5     |
| 5  | 2023-02-07 | 5     |
| 6  | 2023-02-07 | 9     |
| 7  | 2023-02-07 | 9     |
| 8  | 2023-02-07 | 6     |
| 9  | 2023-02-07 | 6     |
| 10 | 2023-02-07 | 5     |
| 11 | 2023-02-07 | 5     |
| 12 | 2023-02-07 | 4     |
| 13 | 2023-02-07 | 4     |
| 14 | 2023-02-07 | 10    |
| 15 | 2023-02-07 | 10    |
| 16 | 2023-02-07 | 7     |
| 17 | 2023-02-07 | 7     |
| 18 | 2023-02-06 | 2     |
| 19 | 2023-02-06 | 2     |
| 20 | 2023-02-06 | 3     |
| 21 | 2023-02-06 | 3     |
| 22 | 2023-02-06 | 8     |
| 23 | 2023-02-06 | 8     |
| 24 | 2023-02-06 | 6     |
| 25 | 2023-02-06 | 6     |
| 26 | 2023-02-06 | 6     |
| 27 | 2023-02-06 | 6     |

### 3. Git-образ проекта [https://github.com/sergicz/ML\\_Adv\\_Diplom.git](https://github.com/sergicz/ML_Adv_Diplom.git)

### 4. Код docker-compose всего проекта и dockerfile Flask-приложения (для запуска потребуется настроить .env-файл по приложенному образцу .env.example, запуск контейнеров через "docker-compose up --build")

[https://github.com/sergicz/ML\\_Adv\\_Diplom/blob/main/docker-compose.yml](https://github.com/sergicz/ML_Adv_Diplom/blob/main/docker-compose.yml)

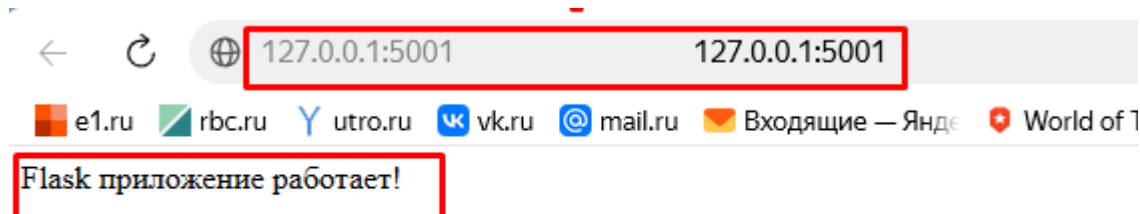
[https://github.com/sergicz/ML\\_Adv\\_Diplom/blob/main/dockerfile](https://github.com/sergicz/ML_Adv_Diplom/blob/main/dockerfile)

## 5. Python-код Flask-приложения

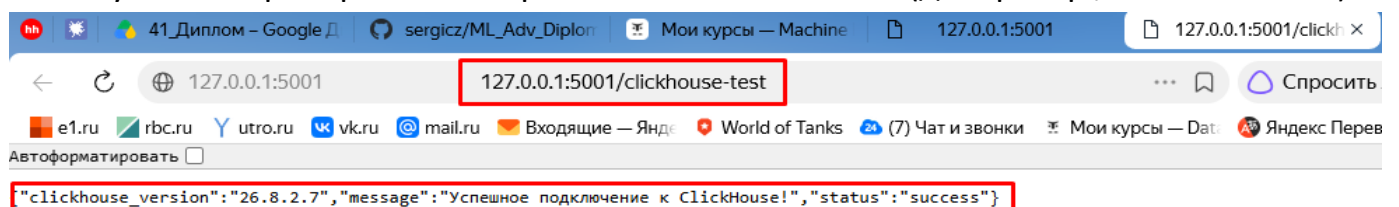
[https://github.com/sergicz/ML\\_Adv\\_Diplom/blob/main/app.py](https://github.com/sergicz/ML_Adv_Diplom/blob/main/app.py)

## 6. Обращение к API приложения по адресу <http://127.0.0.1:5001>

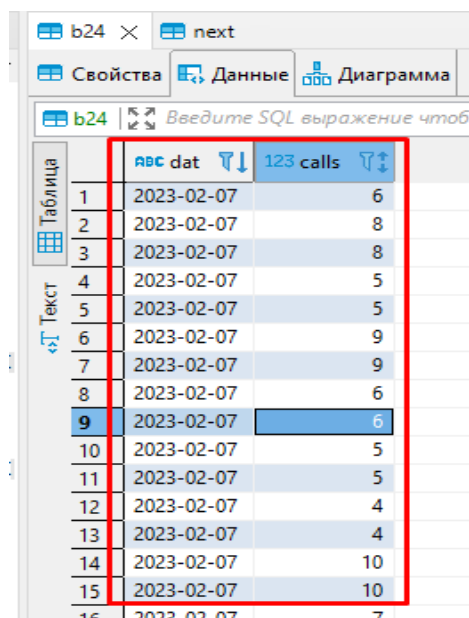
## 7. Результаты HealfCheck приложения (декоратор /)



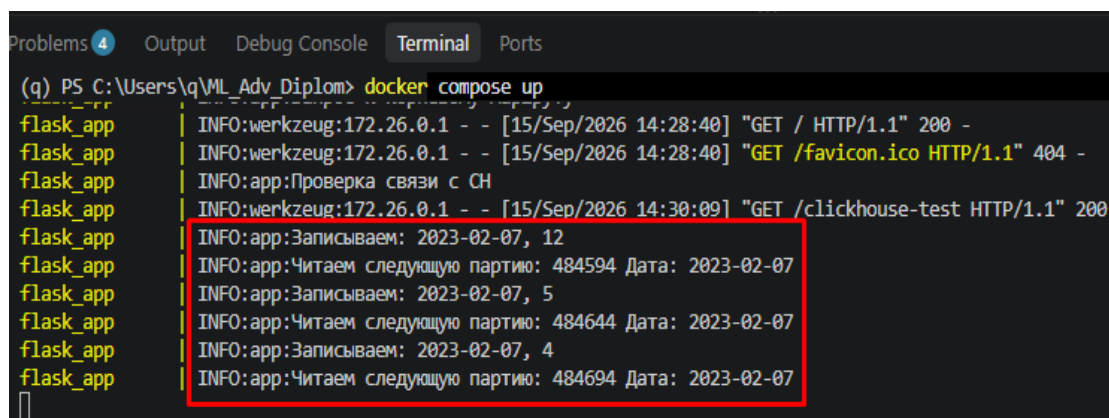
## 8. Результаты проверки связи приложения с ClickHouse (декоратор /clickhouse-test)



## 9. Результаты загрузки данных Flask-Приложением (декоратор /import) + логирование загрузки



|    | ABC dat    | 123 calls |
|----|------------|-----------|
| 1  | 2023-02-07 | 6         |
| 2  | 2023-02-07 | 8         |
| 3  | 2023-02-07 | 8         |
| 4  | 2023-02-07 | 5         |
| 5  | 2023-02-07 | 5         |
| 6  | 2023-02-07 | 9         |
| 7  | 2023-02-07 | 9         |
| 8  | 2023-02-07 | 6         |
| 9  | 2023-02-07 | 6         |
| 10 | 2023-02-07 | 5         |
| 11 | 2023-02-07 | 5         |
| 12 | 2023-02-07 | 4         |
| 13 | 2023-02-07 | 4         |
| 14 | 2023-02-07 | 10        |
| 15 | 2023-02-07 | 10        |





10. Запуск обучения и сохранения модели в joblib (декоратор /train), на вход подается json-файл с максимальной датой TS-ряда

Overview [CONFLICT] GET Get data POST Post data + No environment

REST API basics: CRUD, test & variable > Get data

GET http://127.0.0.1:5001/train Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "target_date": "2023-01-20"
3 }
```

Body Cookies Headers 5 Test Results 1/1 200 OK • 382 ms • 284 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "message": "Модель обучена и сохранена на данных до 2023-01-20",
3   "status": "success"
4 }
```

11. Запуск загрузки сохраненной модели и предсказания количества звонков на определенную дату в будущем (декоратор /predict), на вход подается json-файл с датой (+ выводится фактическое количество звонков при наличии данных)

Overview [CONFLICT] POST Get data POST Post data + No environment

REST API basics: CRUD, test & variable > Get data

POST http://127.0.0.1:5001/predict Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "target_date": "2023-01-30"
3 }
```

Body Cookies Headers 5 Test Results 1/1 200 OK • 253 ms • 246 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "message": "Прогноз на 2023-01-30: 56 факт 47",
3   "status": "success"
4 }
```

### **Используемый технологический стек из курса OTUS “Machine Learning.Advanced”:**

1. Docker
2. Docker-compose
3. Flask-приложение в Docker-контейнере с выносом критичной информации в .env-файл и логированием операций
4. ClickHouse в Docker-контейнере
5. DBeaver
6. Postman
7. Python-библиотеки clickhouse\_connect, joblib, prophet, flask, json, logging, requests.
8. GitHub
9. VSCode
10. Вывод модели в продакшн
11. Временные ряды

### **Выводы из дипломной работы и всего курса**

1. В процессе работы над проектом удалось применить на практике многое из материалов курса ML.Advanced:
  - получен «боевой» опыт создания комплекта микросервисных приложений с использованием docker и docker-compose
  - разработано достаточно сложное Flask-приложение
  - получены навыки работы с Postman
  - написан парсер данных из Битрикс24 с сохранением их в базу ClickHouse
  - построена TS-модель предсказаний на основе библиотеки Prophet от Facebook
  - получен опыт сохранения и повторного использования модели через joblib
  - настроено взаимодействие с Flask-приложением через API
  - выполнено предсказание реальных будущих данных - на их основе могут приниматься кадровые и управленческие решения с пользой для бизнесаСчитаю, что бизнес-цель дипломной работы достигнута на 100%.
2. Материалы курса существенно расширили мои знания в области ML, деплоя моделей в продакшн, работы с временными рядами, построения микросервисной архитектуры, разработки Flask-приложений, Reinforcement Learning, байесовских методов, генерации признаков и многого другого.
3. Сам курс оказался достаточно сложен в изучении, тем не менее удалось пройти его полностью и выполнить все ДЗ, так как в данный момент я не являюсь практикующим ML-инженером. Рассматриваю его как очередную ступеньку к смене сферы деятельности и будущему трудоустройству в области Data Science.